

طراحی کامپوننت Topup در Payment Switch

1. مقدمه

در سیستم Payment Switch، ترمینال‌ها (POS، Payment Gateway و سایر کانال‌ها) درخواست‌های پرداخت را به Switch ارسال می‌کنند. Switch بر اساس نوع تراکنش، عملیات مورد نیاز را انجام می‌دهد. در ابتدا سیستم فقط تراکنش‌های مالی استاندارد مانند:

Purchase

Advice

Reverse

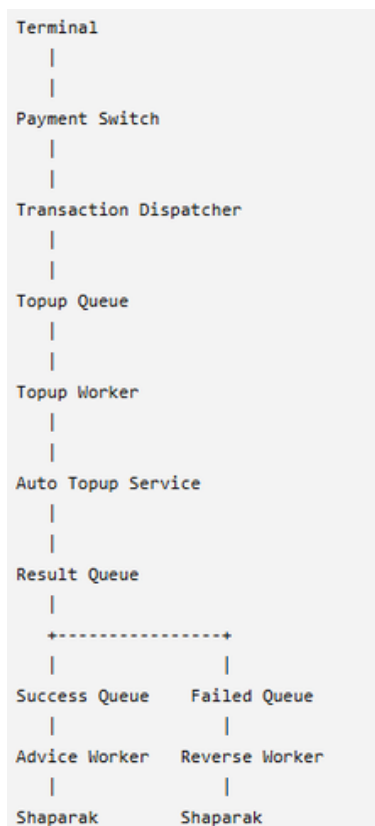
را پشتیبانی می‌کرد.

با اضافه شدن قابلیت شارژ فوری سیم‌کارت (Topup)، نیاز است یک workflow مستقل برای پردازش درخواست‌های شارژ ایجاد شود.

2. معماری کلی

معماری پیشنهادی بر اساس Asynchronous Processing + Queue Based Architecture طراحی شده است.

جریان کلی:



3. مسئولیت Component ها

Payment Switch مسئول

- دریافت تراکنش از ترمینال
- تشخیص نوع تراکنش
- Dispatch کردن درخواست به handler مناسب

برای تراکنش Topup:

Transaction.Type = Topup

درخواست را وارد Topup Queue می‌کند Switch خود عملیات شارژ را انجام نمی‌دهد.

Topup Queue.4

یک صف مستقل برای درخواست‌های شارژ است.

هدف:

- جدا کردن عملیات شارژ از flow اصلی پرداخت
- جلوگیری از block شدن Switch
- امکان retry و مدیریت خطا

نمونه پیام:

```
public class TopupRequest
{
    public string Mobile { get; set; }

    public decimal Amount { get; set; }

    public Guid TransactionId { get; set; }
}
```

Topup Request Worker.5

یک Background Service است که:

- از Topup Queue پیام دریافت می‌کند
- سرویس شارژ خودکار را صدا می‌زند
- نتیجه را بررسی می‌کند

Retry Strategy.6

برای جلوگیری از شکست‌های موقت، مکانیزم Retry طراحی شده است.

- حداکثر 3 تلاش
- بعد از هر شکست، پیام دوباره وارد Queue می‌شود
- بعد از رسیدن به محدودیت Retry، پیام Failed می‌شود

Attempt 1

|

Fail

|

Queue

Attempt 2

|

Fail

|

Queue

Attempt 3

|

Fail

|

Failed Queue

Success / Failed Queue.7

بعد از پردازش تراکنش های شارژ فوری و فراخوانی سرویس شارژ نتیجه در دو صف مجزا درج می شود:

Advice Queue: در صورت موفق بودن عملیات شارژ تراکنش در این صف درج می شود.

:Topup Success Worker

- بازیابی تراکنش از صف
- فراخوانی سرویس Advice

Success Queue

|

|

Advice Handler

|

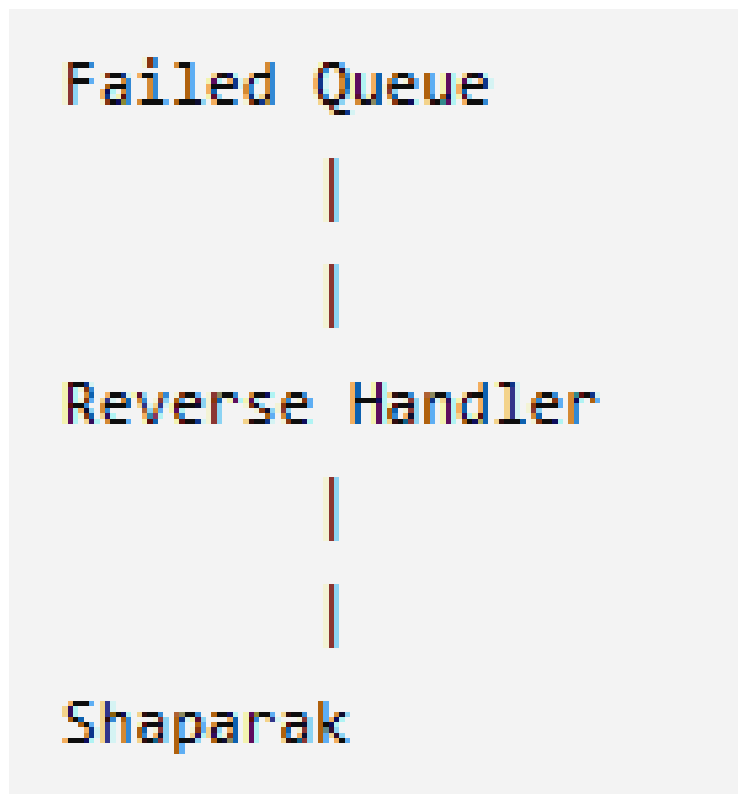
|

Shaparak

Reverse Queue: در صورت ناموفق بودن عملیات شارژ تراکنش در این صف درج می شود.

Topup Failure Worker:

- بازیابی تراکنش از صف
- فراخوانی سرویس Reverse



8.تصمیمات فنی

دلایل استفاده از صف:

- **Decoupling:** کاهش وابستگی بین Payment Switch و سرویس شارژ، به طوری که هر بخش بتواند مستقل توسعه و نگهداری شود.
- **Asynchronous Processing:** جلوگیری از بلاک شدن Payment Switch و افزایش سرعت پاسخدهی به درخواست های ورودی.
- **Retry Mechanism:** امکان تلاش مجدد برای درخواست های ناموفق بدون نیاز به دخالت Payment Switch.
- **Scalability:** امکان افزایش تعداد Worker ها برای پردازش همزمان درخواست های بیشتر بدون ایجاد تغییر در سایر اجزای سیستم.
- **Maintainability:** جداسازی مسئولیت ها و ساده تر شدن توسعه، تست و نگهداری سیستم.

دلایل استفاده از Dispatcher Pattern:

برای هندل کردن تراکنش‌ها بر اساس نوع آن‌ها، از Transaction Dispatcher استفاده شده است.

- حذف شرط‌های متعدد (if/else یا switch) از Payment Switch.
 - امکان افزودن انواع جدید تراکنش (مانند Topup) بدون تغییر در منطق اصلی Switch.
 - رعایت اصل Open/Closed Principle.
 - جداسازی منطق هر نوع تراکنش در Handler مستقل.
-

دلایل استفاده از BackgroundService:

برای پردازش پیام‌های Queue از BackgroundService استفاده شده است.

- پردازش مداوم و غیرهمزمان پیام‌ها.
- پیاده‌سازی ساده Consumerهای Queue.